Module 2

Binary representation of data & numbers

Badharudheen P
Assistant Professor,
Dept. of CSE, MESCE, Kuttippuram

Syllabus

- **Binary representation of data and numbers:** Integer Representation, Data storage units - bits, bytes, kilobytes, etc., ASCII and Unicode**.**
- **CPU Architecture and Instruction Set:** Basic CPU architecture - ALU, registers, control unit, Instruction format and assembly language (basics only) Fetch execute cycle and instruction execution.

Binary Representation

- The binary system is a way of representing data using 0s and 1s.
- This system is used by computers to represent all the data it works with.
- It has base 2 (two base numbers 0 and 1), these base numbers are called bits.
- In binary number system, group of 4 bits is known as Nibble, and a group of 8 bits is known as Byte.
- A thousand bytes form a Kilo-byte (Kb)
- A thousand Kilobytes form a Mega-byte (Mb)
- A thousand Megabytes form a Giga-byte (Gb) and so on (Terabyte (TB), Petabyte (PB), Exabyte (EB) …).

Decimal to Binary Conversion

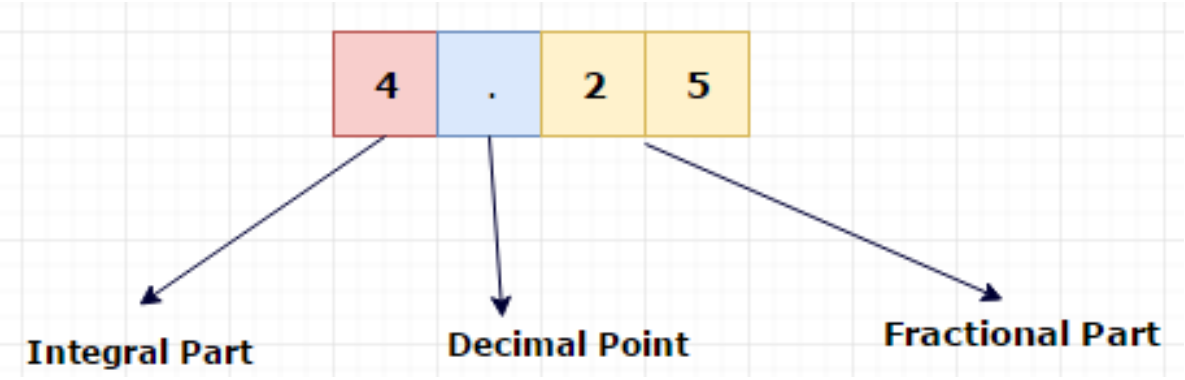
- Repeatedly divide the decimal number by 2 and write down the remainders.
- The remainders, read in reverse order, form the binary representation.
- Steps
  1. Divide the decimal number by 2.
  2. Write down the remainder, which will be either 0 or 1.
  3. Repeat steps 1 and 2 using the quotient from the previous division as the new decimal number.
  4. Continue until the quotient is 0.
  5. Read the remainders in reverse order to get the binary representation.

Decimal to Binary Conversion - Questions

- Convert the following decimal numbers into its equivalent binary.
  - 278
  - 145
  - 362
  - 1126

Floating Point Number to Binary

- To convert an integral part into binary, just follow the previously discussed method.
- To convert the fractional part to binary, multiply fractional part with 2 and take the one bit which appears before the decimal point.
- Follow the same procedure with after the decimal point (.) part until it becomes 1.0.



Floating Point Number to Binary

- Example: Convert 4.25 to binary
  - Integral Part is 4, its binary representation is **$(100)_2$**
  - Fractional part is .25
    - 0.25 * 2 =0.50 //take 0 and move 0.50 to next step.
    - 0.50 * 2 =1.00 //take 1 and stop the process
    - 0.25 = **$(01)_2$**
  - By combining, we get **$(100.01)_2$**

- Note:
  - Some fractional part numbers will not end up with 1.0 with the above method.
  - In floating number storage, the computer will allocate 23 bits for the fractional part. So, it's enough to do the above method at max 23 times.

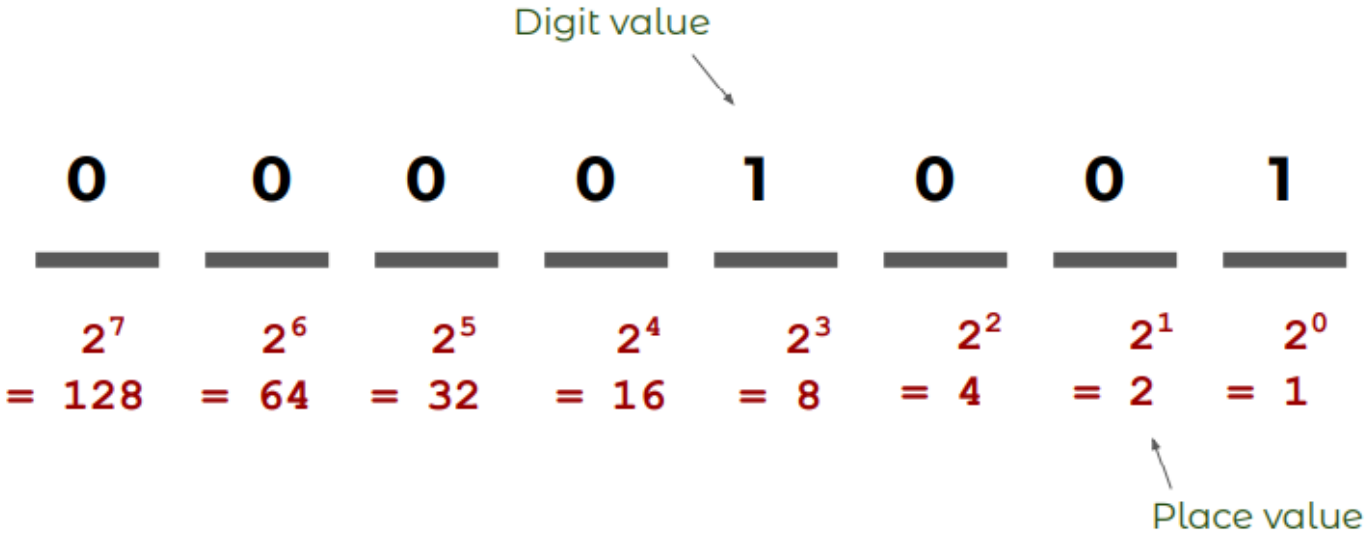
Floating Point to Binary - Questions

- Convert the following floating-point numbers into its equivalent binary.
  - 10.75
  - 2.25
  - 8.33 (fractional part numbers will not end up with 1.0)

Binary to Decimal Conversion

- A binary number is converted to decimal equivalent simply by summing together the weights of various positions in the binary number.

- Example: 00001001



- To compute a number in this system, you would multiply the digit value by the place value, then add them all together: (1*8)+(0*4)+(0*2)+(1*1) = 9

Binary to Decimal Conversion - Questions

- Convert the following binary numbers into its equivalent decimal.
  - 1101
  - 10011101
  - 0010110011
  - 1110110101
  - 10111010

Binary Fraction to Decimal Conversion

A. Convert the integral part of binary to decimal equivalent
   - Multiply each digit separately from left side of radix point till the first digit by 2^0, 2^1, 2^2,… respectively.
   - Add all the result coming from step 1.
   - Equivalent integral decimal number would be the result obtained in step 2.

B. Convert the fractional part of binary to decimal equivalent
   - Divide each digit from right side of radix point till the end by 2^1, 2^2, 2^3, … respectively.
   - Add all the result coming from step 1.
   - Equivalent fractional decimal number would be the result obtained in step 2.

C. Add both integral and fractional part of decimal number.

Binary Fraction to Decimal Conversion

- Given a string of binary number n. Convert binary fractional n into its decimal equivalent.

- Examples:

  Input: n = 110.101

  Output: 6.625

  Input: n = 101.1101

  Output: 5.8125

Binary Fraction to Decimal Conversion - Example

- Let's take an example for n = 110.101

Step 1: Conversion of 110 to decimal

=> $110_2 = (1*2^2) + (1*2^1) + (0*2^0)$

=> $110_2 = 4 + 2 + 0$

=> $110_2 = 6$, so equivalent decimal of binary integral is 6.

Step 2: Conversion of .101 to decimal

=> $0.101_2 = (1*1/2^1) + (0*1/2^2) + (1*1/2^3)$

=> $0.101_2 = 1*0.5 + 0*0.25 + 1*0.125$

=> $0.101_2 = 0.625$, so equivalent decimal of binary fractional is 0.625

Step 3: Add result of step 1 and 2.

=> 6 + 0.625 = 6.625

Binary Fraction to Decimal Conversion - Questions

- Convert the following binary fraction numbers into its equivalent decimal.
  - 11.01
  - 100.11
  - 00101.100
  - 111.011
  - 1011.101

ASCII Code

- **A**merican **S**tandard **C**ode for **I**nformation **I**nterchange.
- It is a character encoding standard that represents text using binary numbers.
- It is used in computers, telecommunications equipment, and other devices.
- uses 7-bit binary numbers to represent 128 characters.
- The first 33 ASCII code points are non-printing control characters, such as the carriage return, line feed, and tab.
- The remaining 95 code points are printable characters, such as letters, numbers, and punctuation.

ASCII Table

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Example

- Encode the word "DATA" and convert the encoded value into binary values which can be understood by a computer.

- Answer:
  - ASCII value of D is 68 and its equivalent 7-bit binary code = 1000100
  - ASCII value of A is 65 and its equivalent 7-bit binary code = 1000001
  - ASCII value of T is 84 and its equivalent 7-bit binary code = 1010100
  - ASCII value of A is 65 and its equivalent 7-bit binary code = 1000001

	D	A	T	A
ASCII Code	68	65	84	65
Binary Code	1000100	1000001	1010100	1000001

Example - Encoding

- Encode the string data "**`Good`**" into its binary representation using ASCII/UTF-8.

  Ans: 01000111 01101111 01101111 01100100

- Encode the string data "**`Hello`**" into its binary representation using ASCII/UTF-8.

  Ans: 01001000 01100101 01101100 01101100 01101111

Example - Decoding

- Decode the binary data "**`01101000 01100001 01101001`**" into its text representation using ASCII/UTF-8.

  Ans: hai

- Decode the binary data "**`01101000 01100101 01101100 01101100 01101111`**" into its text representation using ASCII/UTF-8.

  Ans: hello

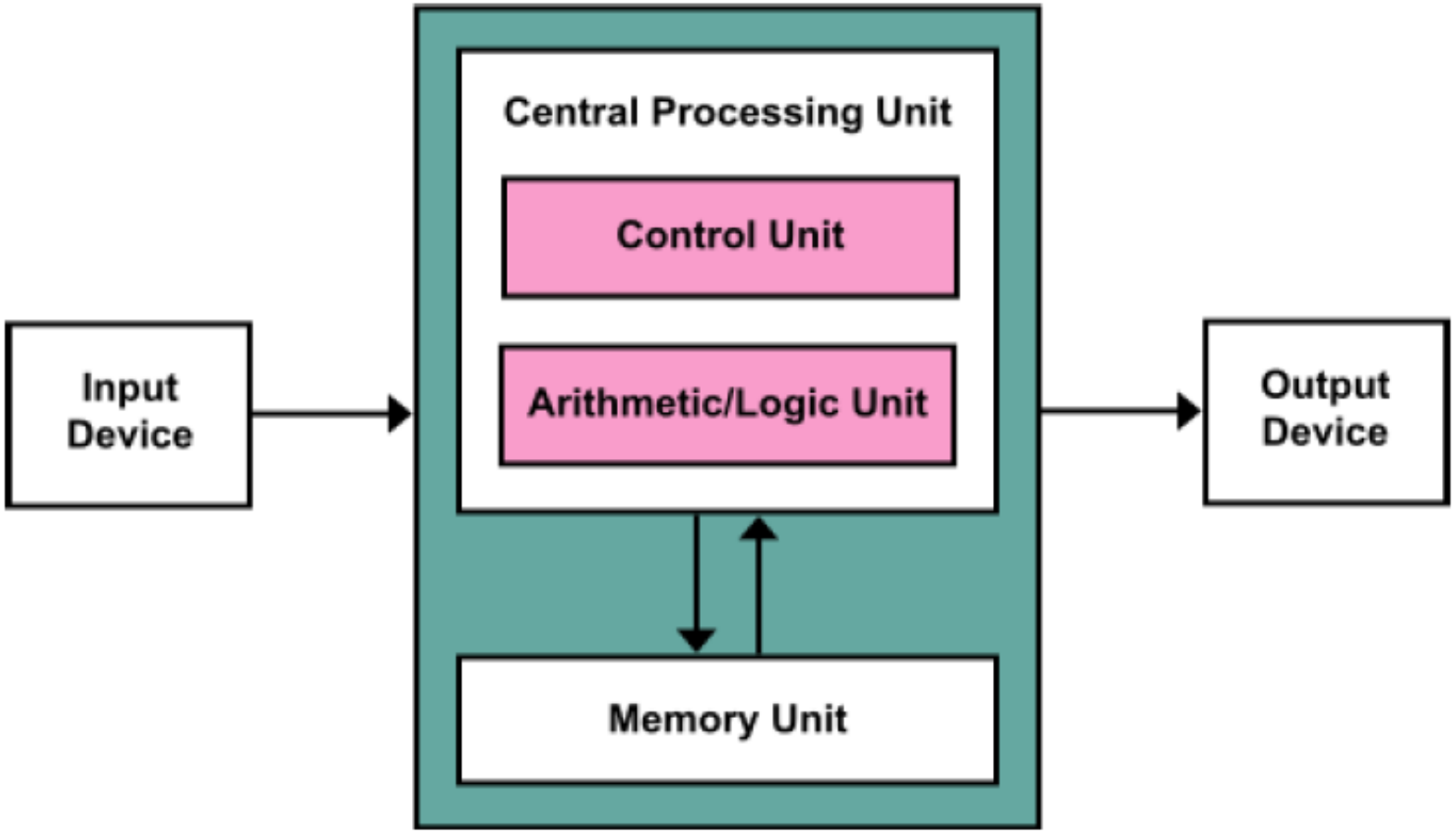
Unicode

- Unicode is a universal character set that defines all the characters needed for writing the majority of living languages in use on computers.
- Unicode is a superset of all other encoded computer character sets.
- The Unicode Standard covers all characters, punctuations, and symbols in the world and enables processing, storage, and transport of text independent of platform and language.
- The most commonly used encodings are UTF-8 and UTF-16 (Unicode Transformation Format).
- UTF-8: a variable-length character encoding (1 to 4 bytes long), is backwards compatible with ASCII and the preferred encoding for e-mail and web pages.
- UTF-16: used in all major operating systems like Windows, IOS, and Unix.

ASCII Vs Unicode

- Both are two popular encoding schemes.
- ASCII
  - 7 bits per character
  - Encodes English letters, numbers and symbols.
  - It is a subset of the Unicode encoding scheme.
  - Requires less space
- Unicode
  - 16, 24 or 32 bits per character
  - Encodes special texts from different languages, letters, symbols, etc.
  - Contains all major languages, symbols, etc.
  - Requires more space.

CPU Architecture



Von Neumann's Computer Architecture Model

CPU

- 3 Main components
  - Control Unit (CU)
  - Arithmetic and Logic Unit (ALU)
  - Memory Unit

Continued…

- Memory: Storage of Information
- Processing Unit: Computation of Information
- Input: Getting information on to the computer
- Output: Getting information out of the computer
- Control Unit: Make sure that all the other parts perform their tasks accurately and at correct time.
- More details already discussed in module 1

CPU Registers

- Also known as a processor register, is a small, fast storage location in a computer's processor that holds data during instruction execution.
- It is internal to the CPU.
- Contains information that the arithmetic and logic unit needs to carry out the current instruction.
- Machine instructions access CPU registers by their addresses.
- In a 64-bit computer, a register must be 64 bits in length.
- The registers can operate in various modes depending on the system architecture, breaking down their storage memory into smaller parts (32-bit into four 8-bit ones, for instance) to which multiple data can be loaded and operated upon at the same time.

Types of CPU Register

- Accumulator: This is the most frequently used register used to store data taken from memory.
- Memory Address Registers (MAR): It holds the address of the location to be accessed from memory.
- Memory Data Registers (MDR): It contains data to be written into or to be read out from the addressed location. MAR and MDR together facilitate the communication of the CPU and the main memory.
- General Purpose Registers: These are numbered as R0, R1, R2….Rn-1, and used to store temporary data during any ongoing operation. Its content can be accessed by assembly programming.

Types of CPU Register

- Program Counter (PC): It is used to keep track of the execution of the program. It contains the memory address of the next instruction to be fetched. The incrementation of PCs depends on the type of architecture being used. If we are using a 32-bit architecture, the PC gets incremented by 4 every time to fetch the next instruction.

- Instruction Register (IR): The IR holds the instruction which is just about to be executed. The instruction from the PC is fetched and stored in IR. As soon as the instruction is placed in IR, the CPU starts executing the instruction, and the PC points to the next instruction to be executed.

Types of CPU Register

- Stack Pointer (SP): The stack PCs pointer points to the top of the stack, which is a part of the memory used to store function calls and other operations.

- Flag Register: A flag register, also known as a status register or condition code register, is a special type of register used to indicate the status of the CPU or the outcome of various operations such as Zero Flag, Carry flag, Sign Flag, Overflow Flag, Parity Flag, Auxiliary Carry Flag, and Interrupt Enable Flag.

Instruction Set

- An instruction set is a collection of commands that a CPU can understand.
- These commands are written in machine language, made up of 1s and 0s.
- Instruction sets tell the CPU how to perform tasks.
- Instructions can be simple commands like read, write, or move.
- Instructions are made up of a specific number of bits.
  - For instance, the CPU's instructions might be 8 bits, where the first 4 bits make up the operation code that tells the computer what to do. The next 4 bits are the operand, which tells the computer the data that should be used.

- Also, the length of an instruction set can vary from as few as 4 bits to many hundreds.

Activities of Instruction Set Commands

- **Data handling and memory management:** used when setting a register to a specific value, copying data from memory to a register or vice versa, and reading and writing data.

- **Arithmetic and logic operations and activities:** These commands include add, subtract, multiply, divide and compare, which examines values in two registers to see if one is greater or less than the other.

- **Control flow activities:** One example is branch, which instructs the system to go to another location and execute commands there. Another is jump, which moves to a specific memory location or address.

Types of Instructions

- **Data Transfer Instructions:** Move data between registers, memory, and input/output devices.
  - MOV – Transfer data from one location to another
  - LOAD – Load data from memory into a register
  - STORE – Store data from a register to memory
  - PUSH/POP – Move data to/from the stack

- **Arithmetic Instructions:** For mathematical operations.
  - ADD, SUB, MUL, DIV, INC/DEC

Types of Instructions

- **Logical Instructions:** Perform bitwise and logical operations.
  - AND – Transfer data from one location to another
  - OR – Load data from memory into a register
  - XOR – Store data from a register to memory
  - NOT– Move data to/from the stack

- **Control Transfer or Branching Instruction:** Used for decision-making and loops.
  - JMP - Jump to another instruction
  - CALL/RET - Call and return from a subroutine
  - JE (Jump if Equal), JNE (Jump if Not Equal), JG (Jump if Greater), JL (Jump if Less).

Types of Instructions

- **Input / Output Instructions:** Allow communication with external devices.
  - IN – Read data from an input port
  - OUT – Send data to an output port

- **Special Purpose Instructions:**
  - NOP (No Operation): CPU does nothing for one cycle.
  - HLT (Halt): Stops CPU execution.

Instruction Format

- Instructions in computers are comprised of groups called fields written in 0s and 1s.
- The most common fields are:
  - The operation field specifies the operation to be performed, like addition.
  - Address field which contains the location of the operand, i.e., register or memory.
  - Mode field which specifies how operand is to be founded (Direct or indirect or register, etc)

Instruction Format

- Based on the number of addresses, instructions are classified as:
  - Zero Address Instructions
  - One Address Instructions
  - Two Address Instructions
  - Three Address Instructions

- **Zero Address Instructions**:
  - Instructions do not specify any operands or addresses. Used in stack-based computers. Operate on data stored in registers or memory locations implicitly defined by the instruction. Ex: PUSH, POP, ADD
  - In the case of ADD, it will take top two elements from stack for the addition and store back to top of the stack.

Instruction Format

- **One Address Instructions**:
  - Instructions specify one operand or address, which refers to a memory location or register, the result may be stored in an accumulator register.
  - It uses an implied ACCUMULATOR register for data manipulation. One operand is in the accumulator and the other is in the register or memory location.

opcode	operand/address of operand	mode

  - Here the mode is the addressing mode used.
  - Example: ADD B

Instruction Format

- **Two Address Instructions**:
  - Instructions specify two operand or address, which refers to a memory location or register, the result may be stored in the same or a different location.

opcode	Destination address	Source address	mode

  - Example: ADD A, B => A = A + B

Instruction Format

- **Three Address Instructions**:
  - Instructions specify three operand or address, which refers to a memory location or register, the result may be stored in the same or a different location.

opcode	Destination address	Source address	Source address	mode

  - Example: ADD A, B, C => A = B + C

Addressing Modes

1. **Immediate Addressing:** Operand is directly specified in the instruction.
   - Example: MOV R1, #5 (Move 5 to R1)
2. **Register Addressing:** Operand is stored in a register.
   - Example: MOV R1, R2 (Copy value of R2 to R1)
3. **Direct Addressing:** Operand is stored in a memory location specified in the instruction.
   - Example: MOV R1, [500H] (Load value from memory location 500H into R1)
4. **Indirect Addressing:** Operand address is stored in a register.
   - Example: MOV R1, [R2] (Load value from memory address stored in R2 into R1)
5. **Indexed Addressing:** The operand address is calculated using a base address and an offset.
   - Example: MOV R1, [R2 + 4]

Types of Instruction Set

- Reduced Instruction Set Computer (RISC)
- Complex Instruction Set Computer (CISC)

CISC

- CISC has a large instruction set for basic and complex instructions.
- An increased number of instructions (200 to 300) results in a complex processor, requires 100,000 transistors.
- Instructions are of variable lengths, using 8, 16 or 32 bits for storage.
- Requires multiple clock cycles to execute an instruction. So, program execution will be slow.
- It has more addressing modes and fewer registers
- Used in laptops and desktop computers.

- Examples: VAX, PDP-11, and the Intel x86/Pentium CPUs.

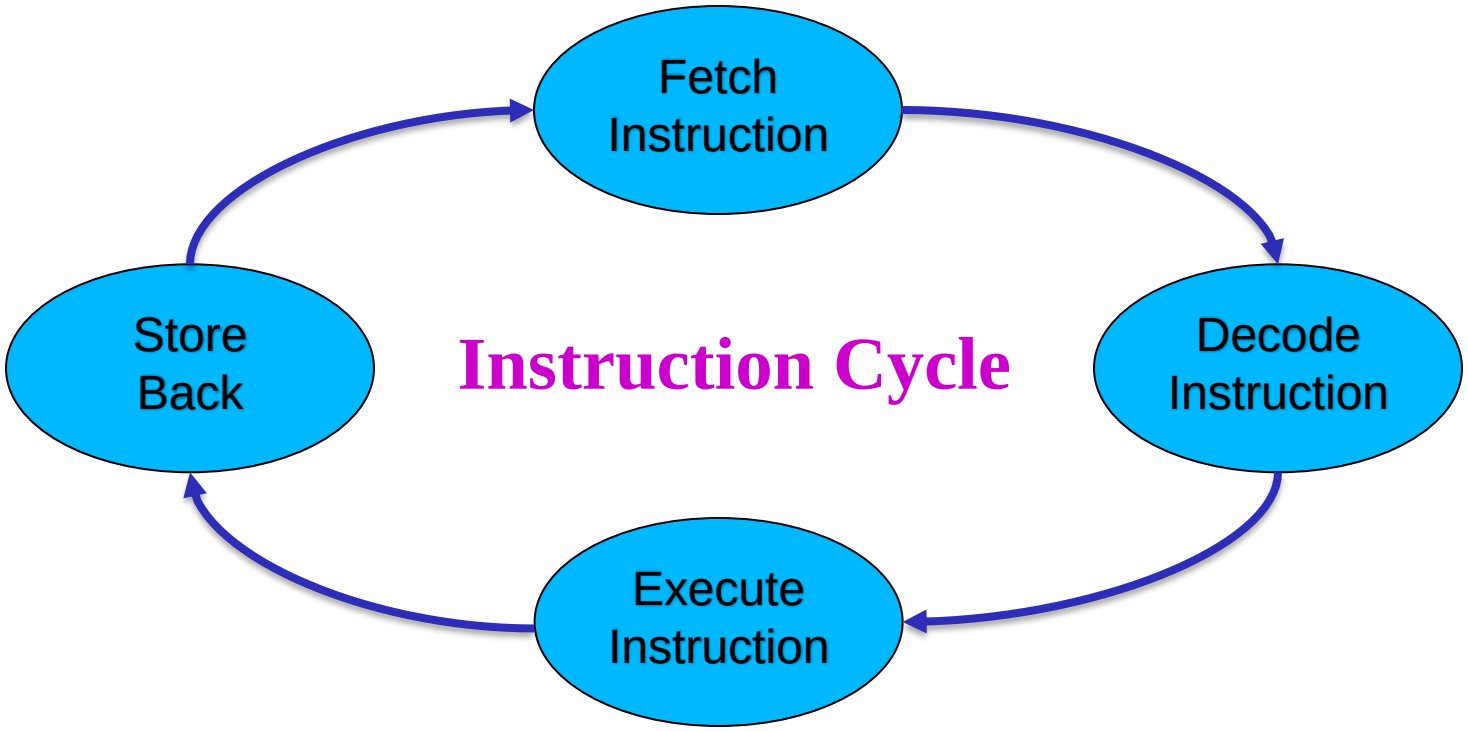
RISC

- RISC has fewer instructions (32 to 47) and requires fewer transistors (44,420), which results in the reduced manufacturing cost of processor.
- The instruction size is fixed (32 bits).
- The instructions, simple in nature, are executed in just one clock cycle, which speeds up the program execution when compared to CISC processors.
- RISC processors can handle multiple instructions simultaneously by processing them in parallel.
- Used in cellphones and tablets.

- Examples: ARM, DEC Alpha, SPARC, MIPS, and PowerPC.

Instruction Execution Cycle

- A program residing in the memory consists of a sequence of instructions.
- These instructions are executed by the processor by going through a cycle for each instruction.



Instruction Execution Cycle

- **Fetch**:
  - The CPU retrieves the instruction from memory.
  - The instruction is stored at the address specified by the program counter (PC).
  - The PC is then incremented to point to the next instruction in memory.

- **Decode**:
  - The CPU interprets the instruction and determines what operation needs to be performed.
  - This involves identifying the opcode and any operands that are needed to execute the instruction.

Instruction Execution Cycle

- **Execute**:
  - The CPU performs the operation specified by the instruction.
  - This may involve reading or writing data from or to memory, performing arithmetic or logic operations on data, or manipulating the control flow of the program.

- **Store Back the Result**:
  - In some CPUs, the results of an instruction are stored during a separate cycle after the execute cycle. This is called the store results cycle.

Assembly Language

- Programming language that overcomes the limitations of machine level language.
- It helps to communicate directly with computer hardware.
- Uses Mnemonic Operation Codes and Symbolic Addresses.
  - For eg: ADD, SUB, MOV, etc.

- Some popular assembly languages include:
  - x86 assembly language, ARM assembly language, MIPS assembly language, PowerPC assembly language, and Little Man Computer (LMC).

Assembly Language

- An instruction consists of three parts:
  - A label - symbolic address of the instruction
  - An operation code - operation to be performed
  - An operand - symbolic address of memory

- Format:
  - **`LABEL: OPERATION CODE OPERAND`**

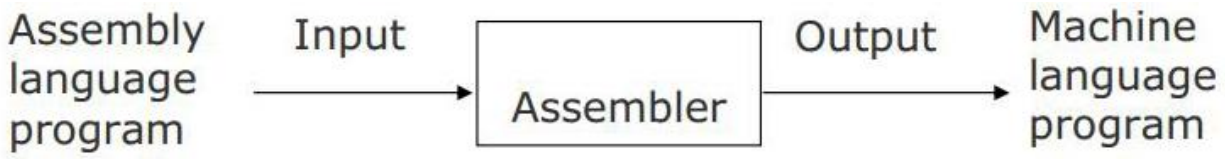
- Eg:- **`ADD A, B`**; Adds the values at the operands addressed by **`A and B`**, result at **`A`**

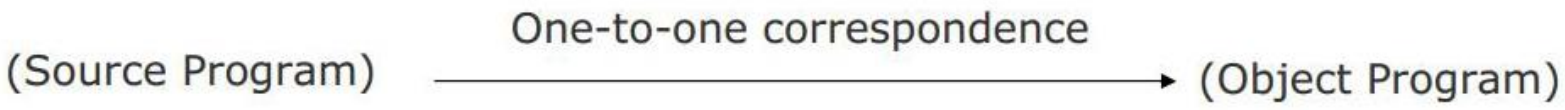
Assembly Language

- Advantages:
  - Easy to understand
  - Easy to locate and correct errors
  - Easy to modify
  - Efficiency almost same as machine language
- Disadvantages:
  - Machine dependent
  - An intermediate translating program is required
  - Cannot be executed in small sized computers
  - Knowledge of hardware required.

Assembler

- Software that translates assembly language program into machine language program

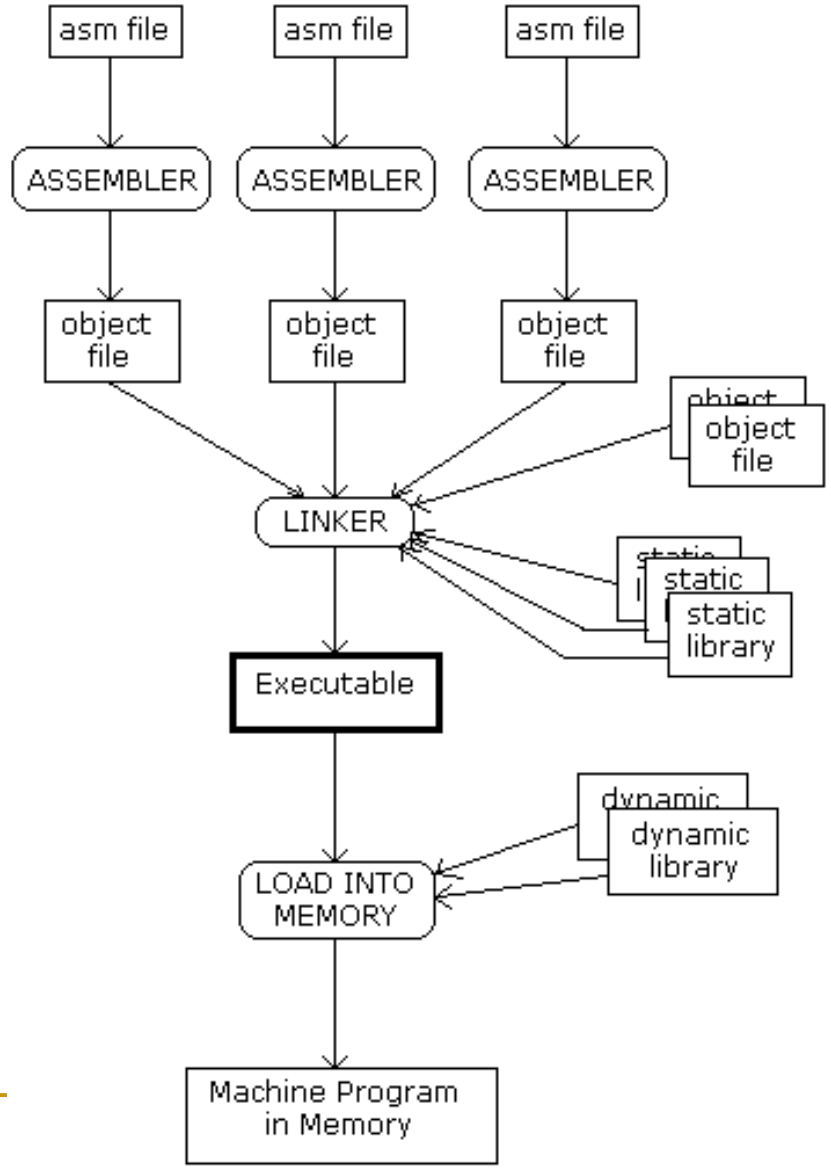




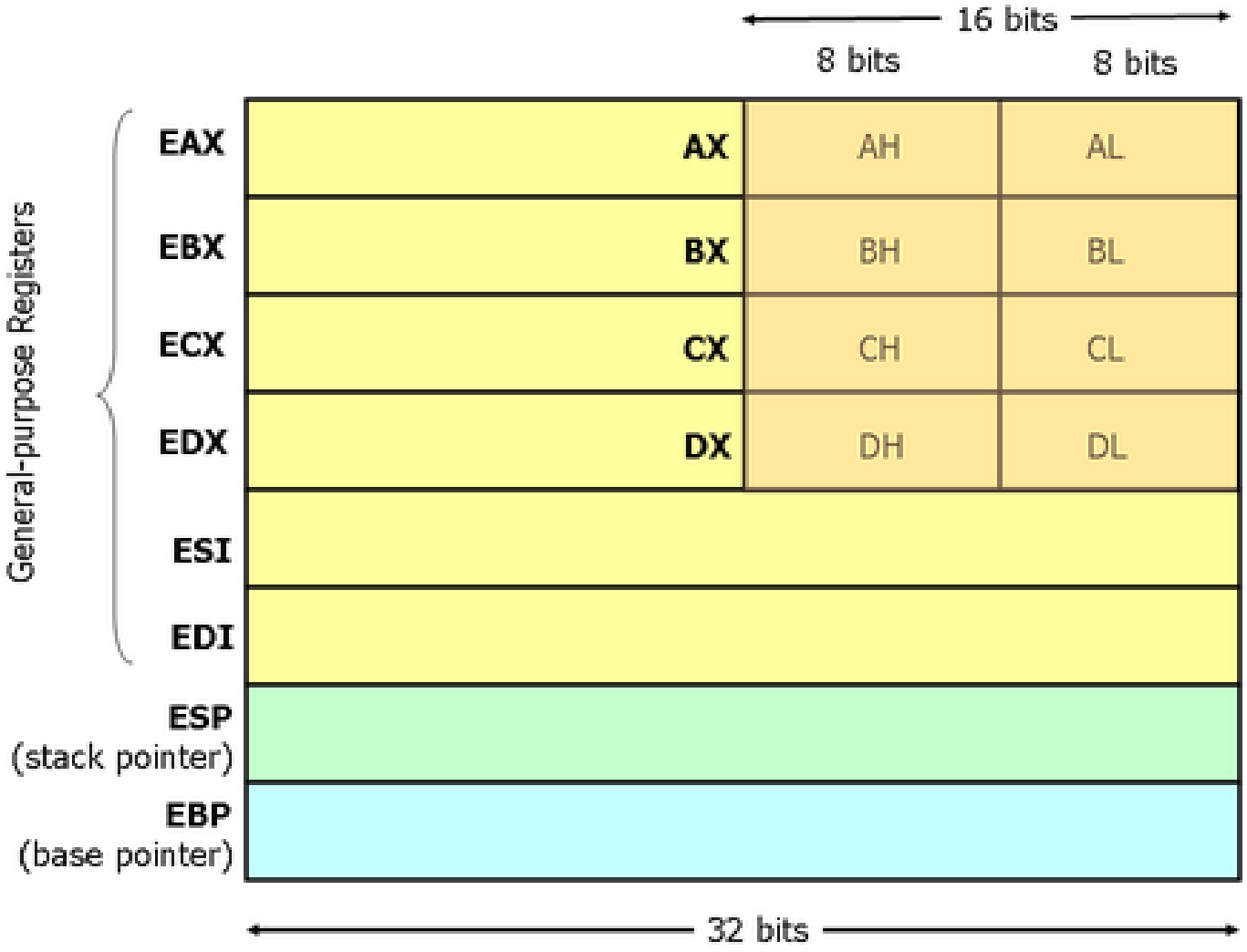
How to execute Assembly Language?

- Write assembly code: Open any text editor and write the mnemonic codes in it and save the file with a proper extension according to your assembler. Extension can be .asm, .s, .asmx.
- Assembling the code: Convert your code to machine language using an assembler.
  - Generating object file: It will generate an object file corresponding to your code. It will have an extension .obj.
  - Linking and creating executables: Assembly language might contain multiple source codes. we have to link them to libraries to make it executable. We can use a linker like lk for this purpose.
- Running program: After creating an executable file we can run it as usual. It will depend on the software that how to run the program.

How to execute Assembly Language?



x86 Registers for Programming



Program to print a message

```
section .data                          ; .data starts here
    msg  db  "Hello World“, 10d        ; String gets initialized
    len equ $-msg                      ; Length of String
section .text                          ; .text starts here
    global _start                      ; Moving to _start
_start:                                ; _start label
    mov eax, 4                         ; Sys_Write Function
    mov ebx, 1                         ; Std_Out File Descriptor
    mov ecx, msg                       ; Offset of msg
    mov edx, len                       ; Length of msg
    int 80h                            ; Call the Kernel
    mov eax, 1                         ; Sys_Exit Function
    mov ebx, 0                         ; Sucessful Termination
    int 80h                            ; Call The Kernel
end:                                   ; end Label
```

Program to add two numbers

```
section .data
    num1 db 5          ; First number
    num2 db 4          ; Second number
    result db 0        ; To store the result

section .text
    global _start

_start:
    mov al, [num1]     ; Load num1 into AL register
    add al, [num2]     ; Add num2 to AL register
    add al, '0'        ; To convert back to ascii
    mov [result], al   ; Store the result
```

```
; Print the result
mov edx, 1
mov ecx, result
mov ebx, 1
mov eax, 4
int 80h

; Exit the program
mov eax, 1
mov ebx, 0
int 80h
```